

Examen SO (22 iunie 2024) Seria A + Seria B

Varianta 1:

Question 1

Incorrect

Mark 0.00 out of 1.00

Flag question

Se execută programul de mai jos, fiind inițializat cu

int n=8; int v[]={6, 2, 7, 9, 2, 4, 8, 7};

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila sum este afișată (la ultima linie) de 0 #4# ori.

Valoarea minimă a sumei afișate este 0 #63#.

Valoarea maximă a sumei afișate este 22 #68#.

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 128KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 64KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	2
7	1
8	9
9	4

Numărul de pagini din memoria virtuală a fiecărui proces este: 128 ✓.

Numărul de cadre din memoria fizică a sistemului este: 64 ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe 17 ✓ biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
8486	9510 ✓
10227	5107 ✓
8025	1881 ✓
2866	-1 ✓

Question 3

Partially correct

Mark 0.31 out of 1.00

Flag question

Se dă un planificator de tipul Shortest Job First, varianta non-preemptivă. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	0	4
2	4	2
3	2	6
4	5	3

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 5 #2#.

0	1 ✓
1	1 ✓
2	1 ✓
3	1 ✓
4	3 #2#
5	3 #2#
6	3 #4#
7	3 #4#
8	3 #4#
9	3 ✓
10	2 #3#
11	2 #3#
12	4 #3#
13	4 #3#
14	4 #3#

Question 4

Incorrect
Mark 0.00 out of 1.00
Flag question

În pseudocodul de mai jos, se consideră împelcările

$n = 8$
 $v = (2, 8, 4, 2, 4, 2, 4, 2, 7, 3)$

Se consideră că indicele din vector încep de la 1.

Funcțiile `MUTEX-LOCK` și `MUTEX-UNLOCK` realizează operațiile respective pe lăcile (mutex-uri).

Funcțiile `CV-WAIT` și `CV-BROADCAST` realizează operațiile respective pe variabile condiționale.

Funcția `CREATE-THREAD` creează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```

THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)
    
```

Pentru fiecare thread ce rulează funcția `THREAD-ALL`, de la 1 la $n-2$, calculați timpul total de rulare (în secunde).

- 21 ☐ 120
22 ☐ 120
23 ☐ 120
24 ☐ 120
25 ☐ 120
26 ☐ 120

Question 5

Incorrect
Mark 0.00 out of 1.00
Flag question

Se rulează codul de mai jos:

```

int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if(++i % 16 == 0) {
        printf("\n");
    }
    printf("\n");
    lseek(fd, 0, SEEK_SET);
    read(fd, &magic, sizeof(magic));
    printf("magic nibble = %d\n", magic % 16);
    read(fd, &size, sizeof(size));
    pos = lseek(fd, size, SEEK_CUR);
    printf("pos = %d\n", pos);
    do {
        if(read(fd, &c, 1) != 1) break;
    } while(c > 0);
    lseek(fd, -2, SEEK_CUR);
    read(fd, &x, 4);
    printf("x = %d\n", x);
    read(fd, &c, 1);
    printf("c = %02x\n", c);
    read(fd, ((unsigned char*)&x + 1), 1);
    printf("x = %d\n", x);
}
    
```

Completați rezultatul afișat pe ecran.

24 58 46 58 18 45 4e 42 43 4b 41 54 59 51 59 46
46 57 55 4a 51 49 57 56 4e 57 51 58 55 42 4e 5a
57 4d 2f 00 00 00 55 03 4c 55 5a 57 58 47 4b 52
49 48 56 4b 59

magic nibble = ☐ [4]

pos = ☐ [29]

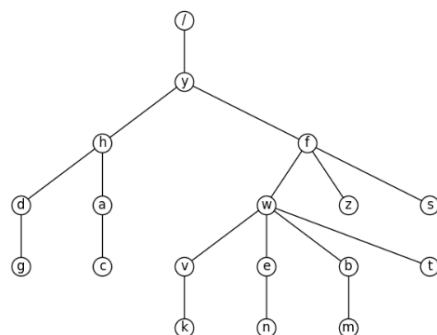
x = ☐ [47]

c = ☐ [55]

x = ☐ [815]

Question 6

Correct
Mark 1.00 out of 1.00
Flag question



Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare. La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea **absolută** spre **e**:

Presupunând că ne aflăm în directorul **/y/f**, completați calea **relativă** spre **v**:

Presupunând că ne aflăm în directorul **/y/f/w/v**, completați calea **relativă** spre **f**:

Presupunând că ne aflăm în directorul **/y/h/d**, completați calea **relativă** spre **b**:

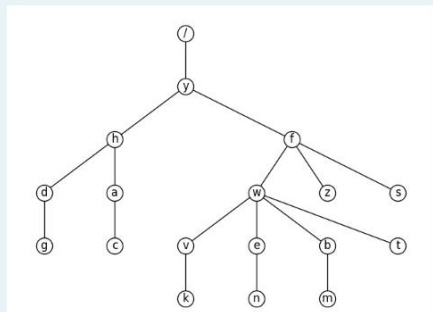
Varianta 2:

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if((++i) % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
lseek(fd, 0, SEEK_SET);
read(fd, &magic, sizeof(magic));
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

96 4b 4a 54 19 49 43 59 4b 47 51 48 4a 41 4f 57
 57 56 4a 4b 5a 45 4d 58 55 58 51 41 57 53 43 46
 46 49 4e 45 51 55 44 1e 00 00 00 58 03 4b 41 52
 44 45 43 43 57 44 4b 55 4e 4e 56 59 59 42
 magic nibble = [6]
 pos = [30]
 x = [30]
 c = [30]
 x = [798]



Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directe.

La completarea câșilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea **absolută** spre e:

Presupunând că ne aflăm în directorul /y/f, completați calea **relativă** spre v:

Presupunând că ne aflăm în directorul /y/f/w/v, completați calea **relativă** spre f:

Presupunând că ne aflăm în directorul /y/h/d, completați calea **relativă** spre b:

În pseudocodul de mai jos, se consideră în tablă:

$n = 7$
 $v = (1, 4, 6, 1, 2, 6, 1)$

Se consideră că indicii din vector încep de la 1.

Funcțiile MUTEX-LOCK și MUTEX-UNLOCK realizează operații respective pe lock (mutex-ul).

Funcțiile CV-WAIT și CV-BROADCAST realizează operații respective pe variabile condiționale.

Funcția CREATE-THREAD creează un thread nou, care primește ca prim argument funcția thread-ului și restul argumentelor se transmit funcției thread-ului.

```
THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   # thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)
```

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1) [12]
 2) [9]
 3) [8]
 4) [8]

Se dă un planificator de tipul **Shortest Job First, varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	0	4
2	4	2
3	2	6
4	5	3

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Desemenea completați aici timpul mediu de așteptare (average waiting time) calculat: ✓.

0	1	✓
1	1	✓
2	1	✓
3	1	✓
4	1	✓
5	1	✓
6	1	✓
7	1	✓
8	1	✓
9	3	✓
10	3	✓
11	3	✓
12	3	✓
13	3	✓
14	3	✓

Se execută programul de mai jos, fiind inițializat cu:

`int n=8; int v[]={6, 2, 7, 9, 2, 4, 8, 7};`

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de ✗ #4# ori.

Valoarea minimă a sumei afișate este ✗ #63#.

Valoarea maximă a sumei afișate este ✗ #68#.

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 4KB. Spațiul virtual de adrese al fiecărui proces este de 512KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 128KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	3
2	6
6	7
9	5

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓.

Numărul de cadre din memoria fizică a sistemului este: ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✗ [19] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
35892	-1 ✓
38041	21657 ✓
39399	23015 ✓
2688	-1 ✗ [14976]

Varianta 3:

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 256KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 512KB. Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	6
4	5
5	3
6	9

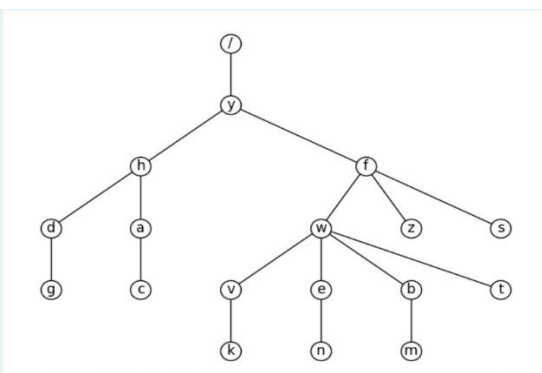
Numărul de pagini din memoria virtuală a fiecărui proces este: 256 ✓.

Numărul de cadre din memoria fizică a sistemului este: 64 ✗ [512].

Adresele virtuale în acest sistem de operare se pot reprezenta pe 18 biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
1004	✗ [7148]
6293	✗ [9365]
6893	✗ [9965]
3551	✗ [-1]



Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare.

La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea **absolută** spre **e**: /y/t/w/e ✓

Presupunând că ne aflăm în directorul /y/t, completați calea **relativă** spre **v**: ./w/v ✗ [w/v/?]

Presupunând că ne aflăm în directorul /y/t/w/v, completați calea **relativă** spre **f**: ../ ✓

Presupunând că ne aflăm în directorul /y/h/d, completați calea **relativă** spre **b**: ../t/w/b ✓

În pseudocodul de mai jos, se consideră inițializarea

$n = 7$

$v = \{0, 5, 2, 3, 6, 1, 4\}$

Se consideră că indicii din vector încep de la 1.

Funcțiile MUTEX-LOCK și MUTEX-UNLOCK realizează operațiile respective pe lacăte (mutex-uri).

Funcțiile CV-WAIT și CV-BROADCAST realizează operațiile respective pe variabile condiționale.

Funcția CREATE-THREAD crează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```

THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)
  
```

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

- 1: 9 ✓
- 2: 10 ✓
- 3: 6 ✓
- 4: 6 ✓
- 5: 11 ✓

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if(++i % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
lseek(fd, 0, SEEK_SET);
read(fd, &magic, sizeof(magic));
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

25 51 4f 4a 13 51 4e 41 48 46 49 4d 55 41 5a 4b
50 4d 44 58 51 57 59 55 5a 42 47 47 48 59 56 56
33 00 00 00 4d 03 59 4e 49 4d 5a 44 41 48 58 58
4b 45 50 4d

magic nibble =

pos = [24]

x =

c =

x =

Se dă un planificator de tipul **Shortest Job First, varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	3	3
2	4	2
3	2	4
4	0	6

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Desenați aici timpul mediu de așteptare (average waiting time) calculat:

0	4 [4]
1	4 [4]
2	4 [4]
3	4 [4]
4	4 [4]
5	4 [4]
6	4 [2]
7	4 [2]
8	4 [1]
9	4 [1]
10	4 [1]
11	4 [3]
12	4 [3]
13	4 [3]
14	4 [3]

Se execută programul de mai jos, fiind inițializat cu

int n=6; int v[]={1, 3, 9, 2, 8, 3};

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de ori.

Valoarea minimă a sumei afișate este .

Valoarea maximă a sumei afișate este .

Varianta 4:

12:10

moodle.cs.utcluj.ro

State

Completed on

Time taken

Marks

Grade

Finished

Saturday, 22 June 2024, 10:10 AM

39 mins 59 secs

2.12/6.00

3.53 out of 10.00 (35.33%)

12:10

moodle.cs.utcluj.ro

Dashboard

My courses

Your answer is partially correct.

Question 1

0.20 out of 1.00

Wrong question

Se rulează codul de mai jos:

```

int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if(++i % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
lseek(fd, 0, SEEK_SET);
read(fd, &magic, sizeof(magic));
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);

```

Completați rezultatul afișat pe ecran.

49 45 48 57 16 50 41 50 46 55 4e 54 45 50 4e 53

44 52 44 41 4c 4c 4a 55 50 56 50 4f 52 47 54 4d

57 24 00 00 00 4d 02 55 45 56 5a 5a 46 4a 53 55

58 52 56 4f 42

magic nibble = 10 ✖ [9]

pos = 4 ✖ [27]

x = 930 ✖ [36]

c = 4d ✔ [36]

x = 2 ✖ [548]

Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare.

La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea absolută spre p: /g/c/p ✔

Presupunând că ne aflăm în directorul /g/c, completați calea relativă spre s: p/s ✔

Presupunând că ne aflăm în directorul /g/c/p/s, completați calea relativă spre c: ../.. ✖ [./././././?]

Presupunând că ne aflăm în directorul /g/r/u, completați calea relativă spre t: ../..../c/p/s/t ✖ [././././c/p/s/t/?]

Se dă un planificator de tipul Shortest Job First, varianta non-preemptivă. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time - AT) și timpul total de rulare (Running Time - RT).

P	AT	RT
1	1	4
2	5	2

12:10

4G+ 70%

moodle.cs.utcluj.ro

moodle-cs-utcluj

Home

Dashboard

My courses

MA

Question 4

Incorrect

Mark 0.00 out of 1.00

Flag question

Se execută programul de mai jos, fiind inițializat cu

int m[] = {1, 3, 9, 2, 8, 3};

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variable «sum» este afișată (la ultima linie) de 12 ✖ #50 ori.

Valoarea minimă a sumei afișate este 26 ✖ #26#.

Valoarea maximă a sumei afișate este 86 ✖ #51#.

Your answer is incorrect.

Question 5

Partially correct

Mark 0.50 out of 1.00

Flag question

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 256KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 512KB.

Se dă tabela de paginare pentru un proces P cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	6
4	5
5	3
6	9

Numărul de pagini din memoria virtuală a fiecărui proces este: 256 ✓

Numărul de cadre din memoria fizică a sistemului este: 512 ✓

Adresele virtuale în acest sistem de operare se pot reprezenta pe 8 ✖ [14] biți.

Comparația mai jos a adrese fizice corespunde cu fiecare adresă virtuală dată. În cazul în care o adresă virtuală nu este mapată completează valoarea -1.

Adresa virtuală	Adresa fizică
1004	7148 ✓
6293	9365 ✓
6893	9965 ✓
3551	-1 ✓

Question 6

Incorrect

Mark 0.00 out of 1.00

Flag question

În pseudocodul de mai jos, se consideră inițializările

n = 8

v = {3, 0, 4, 2, 6, 5, 7, 1}

Se consideră că indicii din vector încep de la 1.

Funcțiile MUTEX-LOCK și MUTEX-UNLOCK realizează operațiile respective pe lock-uri.

Funcțiile CV-WAIT și CV-BROADCAST realizează operațiile respective pe variabile condiționale.

Funcția CREATE-THREAD creează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```
THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)
```

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1:	1	✖	[12]
2:	2	✖	[10]
3:	3	✖	[14]
4:	4	✖	[13]
5:	5	✖	[15]
6:	6	✖	[9]

Your answer is incorrect.

Finish review

12:10

4G+ 70%

Adresa virtuală	Adresa fizică
1004	7148 ✓
6293	9365 ✓
6893	9965 ✓
3551	-1 ✓

În pseudocodul de mai jos, se consideră inițializările

n = 8

v = {3, 0, 4, 2, 6, 5, 7, 1}

Se consideră că indicii din vector încep de la 1.

Funcțiile MUTEX-LOCK și MUTEX-UNLOCK realizează operațiile respective pe lock-uri.

Funcțiile CV-WAIT și CV-BROADCAST realizează operațiile respective pe variabile condiționale.

Funcția CREATE-THREAD creează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```
THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)
```

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1:	1	✖	[12]
2:	2	✖	[10]
3:	3	✖	[14]
4:	4	✖	[13]
5:	5	✖	[15]
6:	6	✖	[9]

Your answer is incorrect.

Finish review

12:10

moodle.cs.utcluj.ro

moodle.cs-utcn

Home

Dashboard

My courses

MA

Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate modulele sunt directoare.

La completarea câmpurilor de mai jos, alegeți tot timpul forma cea mai simplă.

Completat) căsa absolută spre p: /p/0/0 ✓

Preasupunând că ne aflăm în directorul /g/c, completați) căsa relativă spre s: s ✓

Preasupunând că ne aflăm în directorul /g/c/p/a, completați) căsa relativă spre e: ./ ✓

Preasupunând că ne aflăm în directorul /g/c/h, completați) căsa relativă spre t: ././././././././././ ✓

Question 3

Partially correct

Mark 3.0/3.0 out of 5.00

Flag question

Se dă un planificator de tipul Shortest Job First, varianta non-preemptivă. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time - AT) și timpul total de rulare (Running Time - RT).

P	AT	RT
1	1	4
2	5	3
3	0	2
4	3	5

Reprezentăm diagrama Gantt, corelând-o cu exercițiul, completând mai jos cu id-ul procesului activ care selectă pe procesorul la fiecare moment de timp.

Desemnăm completat) aici timpul mediu de așteptare (average waiting time) calculat: 3.75 ✗ [?]

0	3	✓
1	3	✓
2	1	✓
3	1	✓
4	1	✓
5	1	✓
6	4	✗ [?]
7	4	✗ [?]
8	4	✗ [?]
9	4	✓
10	4	✓
11	4	✓
12	2	✗ [?]
13	2	✗ [?]
14	2	✗ [?]

Question 4

Incorrect

Mark 0.0/3.0 out of 5.00

Flag question

Se execută programul de mai jos, fiind inițializat cu:

```
int n=6; int v[7]={1, 3, 9, 2, 8, 3};
```

```
int sum = 0;
for (int i=0; i<n; i++) {
    if (v[i] % 2 == 0) {
        sum += 10;
    }
    if (v[i] % 3 == 1) {
        sum += 0;
    }
}
```

Varianta 5:

Exam seria A, part 2: 40 min; 0/1

moodle.cs.utdcl.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

pt

Question 1

Partially correct

Mark 0.31 out of 1.00

Flag question

Se dă un planificator de tipul **Shortest Job First, varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	0	4
2	4	2
3	2	6
4	5	3

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 3.75 ✖ [2] .

0	1	✓
1	1	✓
2	1	✓
3	1	✓
4	3	✖ [2]
5	3	✖ [2]
6	3	✖ [4]
7	3	✖ [4]
8	3	✖ [4]
9	3	✓
10	2	✖ [3]
11	2	✖ [3]

Quiz navigation

1

2

3

4

5

6

Show one page at a time

Finish review

Search

ENG US

12:41 PM 6/22/2024

Exam seria A, part 2: 40 min: 00

moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

Question 2
Incorrect
Mark 0.00 out of 1.00
Flag question

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if(++i % 16 == 0) {
        printf("\n");
    }
    printf("\n");
    lseek(fd, 0, SEEK_SET);
    read(fd, &magic, sizeof(magic));
    printf("magic nibble = %d\n", magic % 16);
    read(fd, &size, sizeof(size));
    pos = lseek(fd, size, SEEK_CUR);
    printf("pos = %d\n", pos);
    do {
        if(read(fd, &c, 1) != 1) break;
    } while(c > 0);
    lseek(fd, -2, SEEK_CUR);
    read(fd, &x, 4);
    printf("x = %d\n", x);
    read(fd, &c, 1);
    printf("c = %02x\n", c);
    read(fd, ((unsigned char*)&x + 1), 1);
    printf("x = %d\n", x);
}
```

Completați rezultatul afișat pe ecran.

49 45 48 57 16 50 41 50 46 55 4e 54 45 50 4e 53
44 52 44 41 4c 4c 4a 55 50 56 50 4f 52 47 54 4d
57 24 00 00 00 4d 02 55 45 56 5a 5a 46 4a 53 55

Quiz navigation



Show one page at a time

Finish review

Exam seria A, part 2: 40 min: 00

moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

Question 5
Incorrect
Mark 0.00 out of 1.00
Flag question

Se execută programul de mai jos, fiind inițializat cu

```
int n=8; int v[]={6, 2, 7, 9, 2, 4, 8, 7};

int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de ☒ #4# ori.

Valoarea minimă a sumei afișate este ☒ #63#.

Valoarea maximă a sumei afișate este ☒ #68#.

Your answer is incorrect.

Quiz navigation



Show one page at a time

Finish review

Exam seria A, part 2: 40 min: 00

moodle.cs.utdclj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

pt

Question 4
Partially correct
Mark: 0.57 out of 1.00
Flag question

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 256KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 64KB. Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	4
1	6
2	9
8	1

Numărul de pagini din memoria virtuală a fiecărui proces este: 128 ✗ [256].
 Numărul de cadre din memoria fizică a sistemului este: 64 ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe 16 ✗ [18] biți.
 Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
4703	-1 ✓
1637	6757 ✓
9198	16366 ✗ [2030]
2173	9341 ✓

Quiz navigation
 1 2 3 4 5 6
 Show one page at a time
 Finish review

Exam seria A, part 2: 40 min: 00

moodle.cs.utdclj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

pt

THREAD-COUNTER()

```

1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7 return TRUE

```

THREAD-ALL(a_1, a_2, a_3)

```

1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

```

1 val = -1
 2 mutex = MUTEX()
 3 cond = CONDITION-VARIABLE()
 4 for i = 1 to n - 2
 5 // thread number i
 6 CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
 7 CREATE-THREAD(THREAD-COUNTER)

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1: 1 ✗ [7]
 2: 2 ✗ [12]
 3: 3 ✗ [9]
 4: 4 ✗ [13]
 5: 5 ✗ [8]

Quiz navigation
 1 2 3 4 5 6
 Show one page at a time
 Finish review

Exam seria A, part 2: 40 min: 00

moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

```
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

49 45 48 57 16 50 41 50 46 55 4e 54 45 50 4e 53
44 52 44 41 4c 4c 4a 55 50 56 50 4f 52 47 54 4d
57 24 00 00 00 4d 02 55 45 56 5a 5a 46 4a 53 55
58 52 56 4f 42

magic nibble = 0 ✖ [9]

pos = 76 ✖ [27]

x = 19 ✖ [36]

c = 10 ✖ [4d]

x = 0 ✖ [548]

Your answer is incorrect.

Quiz navigation

1 2 3 4 5 6

Show one page at a time

Finish review

Exam seria A, part 2: 40 min: 00

moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187267&cmid=51492

moodle-cs-utcn Home Dashboard My courses

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 3.75 ✖ [2].

0	1	✓
1	1	✓
2	1	✓
3	1	✓
4	3	✖ [2]
5	3	✖ [2]
6	3	✖ [4]
7	3	✖ [4]
8	3	✖ [4]
9	3	✓
10	2	✖ [3]
11	2	✖ [3]
12	5	✖ [3]
13	5	✖ [3]
14	5	✖ [3]

Quiz navigation

1 2 3 4 5 6

Show one page at a time

Finish review

12:47

3G 40

 Review

Question 6

Incorrect

Mark 0.00 out of 1.00

În pseudocodul de mai jos, se consideră inițializările

$n = 7$
 $v = \{3, 4, 0, 5, 2, 6, 1\}$

Se consideră că indicii din vector încep de la 1. Funcțiile `MUTEX-LOCK` și `MUTEX-UNLOCK` realizează operațiile respective pe lacăte (mutex-uri).

Funcțiile `CV-WAIT` și `CV-BROADCAST` realizează operațiile respective pe variabile condiționale.

Funcția `CREATE-THREAD` crează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

<pre> THREAD-COUNTER() 1 while TRUE 2 MUTEX-LOCK(mutex) 3 val = (val + 1) mod n 4 CV-BROADCAST(cond) 5 MUTEX-UNLOCK(mutex) 6 sleep 1 second 7 return TRUE </pre>	<pre> THREAD-ALL(a₁, a₂, a₃) 1 MUTEX-LOCK(mutex) 2 while val ≠ a₁ 3 CV-WAIT(cond, mutex) 4 while val ≠ a₂ 5 CV-WAIT(cond, mutex) 6 while val ≠ a₃ 7 CV-WAIT(cond, mutex) 8 MUTEX-UNLOCK(mutex) 9 return TRUE </pre>
---	--


```

1  val = -1
2  mutex = MUTEX()
3  cond = CONDITION-VARIABLE()
4  for i = 1 to n - 2
5    // thread number i
6    CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7  CREATE-THREAD(THREAD-COUNTER)

```



Pentru fiecare thread ce rulează funcția `THREAD-ALL` (de la 1 la $n-2$), calculați timpul total de rulare

Show one page at a time

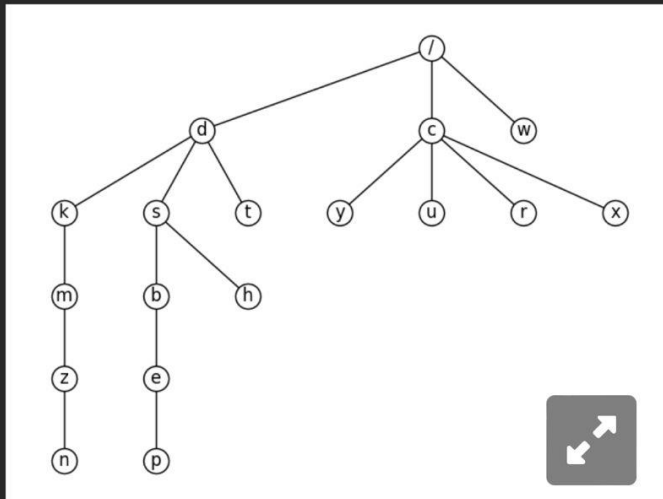
12:47

3G 40

< Review



Question 5

Partially correct
Mark 0.25 out of 1.00

Se dă sistemul de fișiere din figura de mai sus.
 Puteți considera că toate nodurile sunt directoare.
 La completarea căilor cerute mai jos, alegeți tot
 timpul forma cea mai simplă.

Completați calea **absolută** spre **m**:



Presupunând că ne aflăm în directorul **/d**,
 completați calea **relativă** spre **e**:



[s/b/e/?]

Presupunând că ne aflăm în directorul **/d/s/b/e**,
 completați calea **relativă** spre **d**:



[../../?]

Presupunând că ne aflăm în directorul **/d/s/b**,
 completați calea **relativă** spre **z**:



[../..k/m/z/?]

12:44

3G 42

< Review

8	9
9	4

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓.

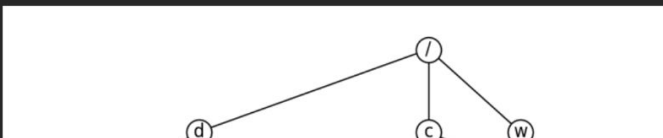
Numărul de cadre din memoria fizică a sistemului este: ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✗ [17] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
8486	8510 ✗ [9510]
10227	5107 ✓
8025	1881 ✓
2866	-1 ✓

Question 5

Partially correct
Mark 0.25 out of 1.00

12:44

3G 42

< Review



Question 4

Partially correct

Mark 0.71 out of 1.00

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 128KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 64KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	2
7	1
8	9
9	4

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓.

Numărul de cadre din memoria fizică a sistemului este: ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✗ [17] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
8486	8510 ✗ [9510]
	5407 ✓

12:44

3G 42

Review

Question 3

Mark 1.00 out of 1.00

Se dă un planificator de tipul **Shortest Job First**, **varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	3	3
2	4	2
3	2	4
4	0	6

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: ✓

0	4 ✓
1	4 ✓
2	4 ✓
3	4 ✓
4	4 ✓
5	4 ✓
6	2 ✓

12:44

3G 42

< Review



Question 2

Partially correct

Mark 0.60 out of 1.00

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if((++i) % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
lseek(fd, 0, SEEK_SET);
read(fd, &magic, sizeof(magic));
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

19 41 41 58 18 41 4b 53 42 4e 4b 4e 45
5a 4b 52 4c 4d 49 4e 52 42 56 50 43 51
44 48 5a 5a 55 2a 00 00 00 43 02 55 4d
50 44 54 4c 53 47 4f 47 52

magic nibble = ✖ [9]

12:44

3G 42

< Review

```
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de

✗ #8# ori.

Valoarea minimă a sumei afișate este

✗

#65# .

Valoarea maximă a sumei afișate este

✗

#74# .

Your answer is incorrect.

Question 2

Partially correct

Mark 0.60 out of 1.00

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
```

12:44

3G 42

Review

```
do {  
    if(read(fd, &c, 1) != 1) break;  
} while(c > 0);  
lseek(fd, -2, SEEK_CUR);  
read(fd, &x, 4);  
printf("x = %d\n", x);  
read(fd, &c, 1);  
printf("c = %02x\n", c);  
read(fd, ((unsigned char*)&x + 1), 1);  
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

```
19 41 41 58 18 41 4b 53 42 4e 4b 4e 45  
5a 4b 52 4c 4d 49 4e 52 42 56 50 43 51  
44 48 5a 5a 55 2a 00 00 00 43 02 55 4d  
50 44 54 4c 53 47 4f 47 52
```

magic nibble = ✗ [9]

pos = ✓

x = ✓

c = ✓

x = ✗ [554]

Your answer is partially correct.

Question 3

Correct
Mark 1.00 out of 1.00

Show one page at a time

12:44

3G 42

Review

Question 1

Mark 0.00 out of 1.00

Se execută programul de mai jos, fiind inițializat cu

```
int n=8; int v[]={9, 4, 3, 2, 9, 2, 2,
```

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de

✗ #8# ori.

Valoarea minimă a sumei afișate este ✗

#65# .

Valoarea maximă a sumei afișate este ✗

12:44

3G 42

< Review



0	4	✓
1	4	✓
2	4	✓
3	4	✓
4	4	✓
5	4	✓
6	2	✓
7	2	✓
8	1	✓
9	1	✓
10	1	✓
11	3	✓
12	3	✓
13	3	✓
14	3	✓

12:47

3G 40

Review

ului iar restul argumentelor se transmit funcției thread-ului.

```

THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER,

```

```

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

```

Pentru fiecare thread ce rulează funcția `THREAD-ALL` (de la 1 la $n-2$), calculați timpul total de rulare (în secunde).

- 1: ✗ [7]
- 2: ✗ [12]
- 3: ✗ [9]
- 4: ✗ [13]
- 5: ✗ [8]

Your answer is incorrect.

Show one page at a time

Varianta 7:

12:22 moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187231&cmid=51492

moodle-cs-utcn Home Dashboard My courses

THREAD-COUNTER()

```

1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7 return TRUE

```

THREAD-ALL(a_1, a_2, a_3)

```

1 MUTEX-LOCK(mutex)
2 while val  $\neq$   $a_1$ 
3   CV-WAIT(cond, mutex)
4 while val  $\neq$   $a_2$ 
5   CV-WAIT(cond, mutex)
6 while val  $\neq$   $a_3$ 
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

```

12:22 moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187231&cmid=51492

moodle-cs-utcn Home Dashboard My courses

```

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7 CREATE-THREAD(THREAD-COUNTER)

```

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

12:23 moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=187231&cmid=51492

moodle-cs-utcn Home Dashboard My courses

7 CREATE-THREAD(THREAD-COUNTER)

Pentru fiecare thread ce rulează funcția THREAD-ALL (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1: [15]

2: [8]

3: [14]

4: [11]

5: [13]

6: [9]

12:22

88%



moodle.cs.utcluj.ro/mc



NK



```

        printf("\n");
    }
}
printf("\n");
lseek(fd, 0, SEEK_SET);
read(fd, &magic, sizeof(magic));
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);

```

Completați rezultatul afișat pe ecran.

```

↓ 58 46 58 18 45 4e 42 43 4b 41 54 59 51 59 46
↓ 57 55 4a 51 49 57 56 4e 57 51 58 55 42 4e 5a
↓ 4d 2f 00 00 00 55 03 4c 55 5a 57 50 47 4b 52
↓ 48 56 4b 59

```

magic nibble = ✓

pos = ✗ [29]

= ✗ [47]

= ✗ [55]

= ✗ [815]



Your answer is partially correct.



12:21

88%



moodle.cs.utcluj.ro/mc



NK



Se execută programul de mai jos, fiind inițializat cu

int n=8; int v[]={2, 1, 2, 9, 5, 5, 9, 5};

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de

✗ #4# ori.

Valoarea minimă a sumei afișate este ✗

#69# .

Valoarea maximă a sumei afișate este ✗

#71# .



Your answer is incorrect.



12:22

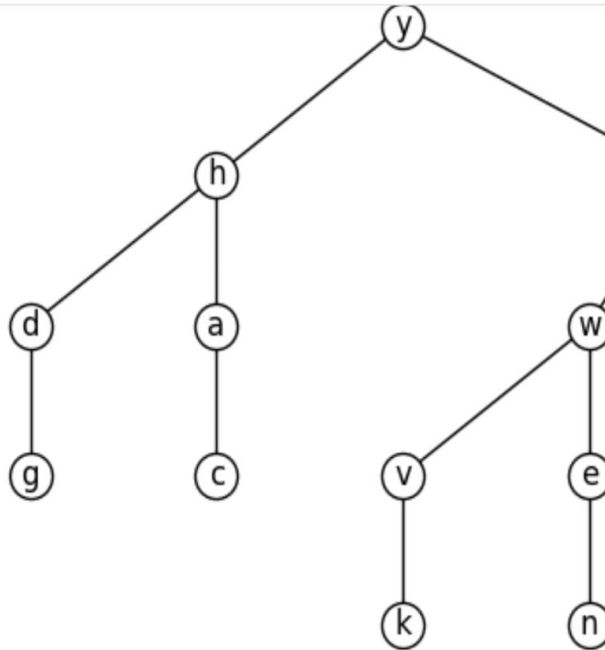
88%



moodle.cs.utcluj.ro/mc



NK



Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare.

La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea **absolută** spre e:

 ✓

Presupunând că ne aflăm în directorul **/y/f**, completați

calea **relativă** spre v: ✓

Presupunând că ne aflăm în directorul **/y/f/w/v**,

completați calea **relativă** spre f:

 ✓

Presupunând că ne aflăm în directorul **/y/h/d**,

completați calea **relativă** spre b:

 ✓

12:21

88%



moodle.cs.utcluj.ro/mc



NK



lucrând cu pagini de 4KB. Spațiul virtual de adrese al fiecărui proces este de 512KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 128KB. Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	9
3	5
4	8
6	3

Numărul de pagini din memoria virtuală a fiecărui proces este: 128 ✓

Numărul de cadre din memoria fizică a sistemului este: 32 ✓

Adresele virtuale în acest sistem de operare se pot reprezenta pe 19 ✓ biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
5806	-1 ✓
16691	33075 ✓
2706	20898 ✓
14794	22986 ✓



Varianta 8:

12:26

1:08



41



Review



Question 2

Correct

Mark 1.00 out of 1.00

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 256KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 512KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	6
4	5
5	3
6	9

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓.

Numărul de cadre din memoria fizică a sistemului este: ✓.

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✓ biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
-----------------	---------------

Show one page at a time

12:26

1:09



< Review



este: 512 ✓ .

Adresele virtuale în acest sistem de operare se pot reprezenta pe 18 ✓ biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
1004	7148 ✓
6293	9365 ✓
6893	9965 ✓
3551	-1 ✓

Question 3

Partially correct
Mark 0.63 out of 1.00

Se dă un planificator de tipul **Shortest Job First, varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	1	3
2	5	4
3	3	2
4	0	6

12:26

1:18



41



Review



Se executa programul de mai jos, fiind initializat
cu

```
int n=6; int v[]={1, 3, 9, 2, 8, 3};
```

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += 10;
        if(v[i] % 3 == 1) {
            exit(0);
        }
    } else {
        sum += v[i];
        if(v[i] % 3 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de

✗ #8# ori.

Valoarea minimă a sumei afișate este ✗

#36#.

Valoarea maximă a sumei afișate este ✗

#51#.

Your answer is incorrect.

Review

inițializările

$n = 7$
 $v = \{0, 5, 2, 3, 6, 1, 4\}$

Se consideră că indicii din vector încep de la 1.

Funcțiile `MUTEX-LOCK` și `MUTEX-UNLOCK` realizează operațiile respective pe lacăte (mutex-uri).

Funcțiile `CV-WAIT` și `CV-BROADCAST` realizează operațiile respective pe variabile condiționale.

Funcția `CREATE-THREAD` crează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```

THREAD-COUNTER()
1  while TRUE
2    MUTEX-LOCK(mutex)
3    val = (val + 1) mod n
4    CV-BROADCAST(cond)
5    MUTEX-UNLOCK(mutex)
6    sleep 1 second
7  return TRUE

THREAD-ALL(a1, a2, a3)
1  MUTEX-LOCK(mutex)
2  while val ≠ a1
3    CV-WAIT(cond, mutex)
4  while val ≠ a2
5    CV-WAIT(cond, mutex)
6  while val ≠ a3
7    CV-WAIT(cond, mutex)
8  MUTEX-UNLOCK(mutex)
9  return TRUE

1  val = -1
2  mutex = MUTEX()
3  cond = CONDITION-VARIABLE()
4  for i = 1 to n - 2
5    // thread number i
6    CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7  CREATE-THREAD(THREAD-COUNTER)

```

Pentru fiecare thread ce rulează funcția `THREAD-ALL` (de la 1 la $n-2$), calculați timpul total de rulare (în secunde).

1: ✗ [9]

2: ✗ [10]

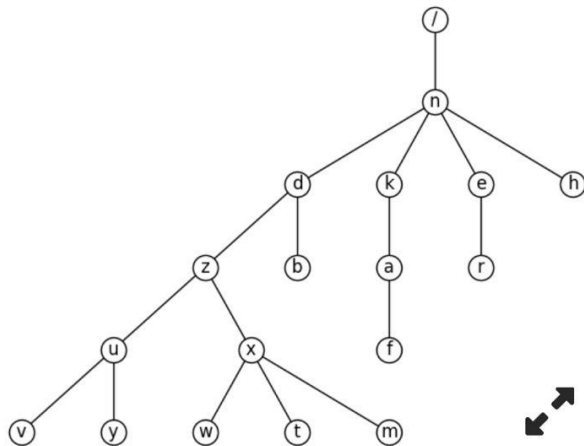
3: ✗ [6]

12:26

1:21

41

Review



Se dă sistemul de fișiere din figura de mai sus.
 Puteți considera că toate nodurile sunt directoare.
 La completarea căilor cerute mai jos, alegeți tot
 timpul forma cea mai simplă.
 Completați calea **absolută** spre **z**:



Presupunând că ne aflăm în directorul **/n/d**,
 completați calea **relativă** spre **x**:



Presupunând că ne aflăm în directorul **/n/d/z/x**,
 completați calea **relativă** spre **d**:



Presupunând că ne aflăm în directorul **/n/k/a**,
 completați calea **relativă** spre **u**:



Question 6

Incorrect

12:26

1:13



41



Review



de rulare (Running Time -- RT).

P	AT	RT
1	1	3
2	5	4
3	3	2
4	0	6

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: ✓.

0	0	✗ [4]
1	0	✗ [4]
2	0	✗ [4]
3	0	✗ [4]
4	0	✗ [4]
5	0	✗ [4]
6	3	✓
7	3	✓
8	1	✓

Show one page at a time

12:26

1:15



41



Review



2	0	✗ [4]
3	0	✗ [4]
4	0	✗ [4]
5	0	✗ [4]
6	3	✓
7	3	✓
8	1	✓
9	1	✓
10	1	✓
11	2	✓
12	2	✓
13	2	✓
14	2	✓

Question 4

Incorrect

Mark 0.00 out of 1.00

Se execută programul de mai jos, fiind inițializat

12:25

1:01



41



Review



Started on

Saturday, 22 June 2024, 9:30 AM

State

Finished

Completed on

Saturday, 22 June 2024, 10:10 AM

Time taken

40 mins

Marks

2.38/6

Grade

3.96 out of 10 (39.58%)

Question 1

Incorrect

Mark 0.00 out of 1.00

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned int magic = 0, x = 0;
unsigned char size = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if((++i) % 16 == 0) {
        printf("\n");
    }
}
```

Show one page at a time

12:25

1:05



41



Review



```
printf("magic nibble = %d\n", magic % 16);
read(fd, &size, sizeof(size));
pos = lseek(fd, size, SEEK_CUR);
printf("pos = %d\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
read(fd, &c, 1);
printf("c = %02x\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = %d\n", x);
```

Completați rezultatul afișat pe ecran.

```
19 41 41 58 18 41 4b 53 42 4e 4b 4e 45
5a 4b 52 4c 4d 49 4e 52 42 56 50 43 51
44 48 5a 5a 55 2a 00 00 00 43 02 55 4d
50 44 54 4c 53 47 4f 47 52
```

magic nibble = ✖ [9]

pos = ✖ [29]

x = ✖ [42]

c = ✖ [43]

x = ✖ [554]

Your answer is incorrect.

Question 2

Correct
Mark 1.00 out of 1.00

Varianta 9:

Question 2
Correct
Mark 1.00 out of 1.00
Flag question

Se dă un planificator de tipul **First Come, First Served**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AI	KI	RI
1	2	2	
2	0	2	
3	3	3	
4	4	4	

Reconstruiți diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesorul la fiecare moment de timp. (Înregistrarea completată) Aici timpul mediu de așteptare (average waiting time) calculat: 6.0.

0	2	✓
1	2	✓
2	2	✓
3	3	✓
4	3	✓
5	4	✓
6	1	✓
7	1	✓
8	2	✓
9	2	✓
10	2	✓
11	4	✓
12	1	✓
13	4	✓
14	4	✓

Quiz navigation
1 2 3 4 5 6
Show one page at a time
Finish review

4-10 martie

moodle-cs-utcn Home Dashboard My courses

Time taken 40 mins
Marks 2.45/6.00
Grade 4.08 out of 10.00 (40.83%)

Question 1
Incorrect
Mark 0.00 out of 1.00
Flag question

Se execută programul de mai jos, fiind inițializat cu
`int n=6; int v[]={3, 9, 6, 1, 8, 6};`

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += v[i];
        if(v[i] % 4 == 1) {
            exit(0);
        }
    } else {
        sum += 12;
        if(v[i] % 4 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila sum este afișată (la ultima linie) de 5 ori. #4# ori.

Valoarea minimă a sumei afișate este 7. #47#.

Valoarea maximă a sumei afișate este 36. #60#.

Your answer is incorrect.

Quiz navigation
1 2 3 4 5 6
Show one page at a time
Finish review

moodle-cs-utcn Home Dashboard My courses

Question 3
Partially correct
Mark 0.25 out of 1.00
Flag question

```
graph TD
    I --> g
    I --> e
    g --> p
    g --> x
    p --> t
    p --> b
    p --> h
    p --> z
    t --> f
    t --> y
    f --> n
    n --> r
    b --> k
    k --> s
```

Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare.
La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea absolută spre b: /g/p/b ✓

Presupunând că ne aflăm în directorul /g/p/t, completați calea relativă spre n: /f/n ✗ [f/n?]

Presupunând că ne aflăm în directorul /g/p/t/n, completați calea relativă spre t: /../ ✗ [../?]

Presupunând că ne aflăm în directorul /g/p/t/f, completați calea relativă spre k: /../b/k ✗ [../b/k?]

Quiz navigation
1 2 3 4 5 6
Show one page at a time
Finish review

moodle-cs-utcn Home Dashboard My courses

Incorrect
Mark 0.00 out of 1.00
Flag question

Regulament de etică profe...
Cursuri - printer friendly
Indrumator - temp

Examen

Exam seria A, part 1; 1...
Exam seria A, part 2; 4...
Exam seria B, part 1; 15 mi...
Exam seria B, part 2; 40 mi...
Exam, part 3; 40 min; 1...

26 februarie - 3 martie

Curs 1: Intro
Curs 1: Quiz A
Curs 1: Quiz B
Lab 1: Instalarea și primii p...
Lab 1: Quiz
Teme: accesul prin GitHub
Teme: GitHub username
Setup VM VirtualBox

4-10 martie

$n = 8$
 $v = \{7, 5, 8, 4, 1, 3, 6, 2\}$

Se consideră că indicii din vector încep de la 1.
Funcțiile `MUTEX-LOCK` și `MUTEX-UNLOCK` realizează operațiile respective pe lacăte (mutex-uri).
Funcțiile `CV-WAIT` și `CV-BROADCAST` realizează operațiile respective pe variabile condiționale.
Funcția `CREATE-THREAD` creează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

```
THREAD-COUNTER()
1 while TRUE
2   MUTEX-LOCK(mutex)
3   val = (val + 1) mod n
4   CV-BROADCAST(cond)
5   MUTEX-UNLOCK(mutex)
6   sleep 1 second
7   return TRUE

THREAD-ALL(a1, a2, a3)
1 MUTEX-LOCK(mutex)
2 while val ≠ a1
3   CV-WAIT(cond, mutex)
4 while val ≠ a2
5   CV-WAIT(cond, mutex)
6 while val ≠ a3
7   CV-WAIT(cond, mutex)
8 MUTEX-UNLOCK(mutex)
9 return TRUE

1 val = -1
2 mutex = MUTEX()
3 cond = CONDITION-VARIABLE()
4 for i = 1 to n - 2
5   // thread number i
6   CREATE-THREAD(THREAD-ALL, v[i], v[i + 1], v[i + 2])
7   CREATE-THREAD(THREAD-COUNTER)
```

Pentru fiecare thread ce rulează funcția `THREAD-ALL` (de la 1 la n-2), calculați timpul total de rulare (în secunde).

1: 6 [16]
2: 5 [12]
3: 1 [9]
4: 4 [11]
5: 2 [6]
6: 1 [10]

Quiz navigation
1 2 3 4 5 6
Show one page at a time
Finish review

Snipping Tool
Screenshot copied to clipboard and saved
Select here to mark up and share.

moodle-cs-utcn Home Dashboard My courses

moodle-cs-utcn Home Dashboard My courses

Exam seria B, part 2; 40 min; 10:50 - 11:35

Started on Saturday, 22 June 2024, 10:50 AM
State Finished
Completed on Saturday, 22 June 2024, 11:30 AM
Time taken 40 mins
Marks 2.45/6.00
Grade 4.08 out of 10.00 (40.83%)

Question 1
Incorrect
Mark 0.00 out of 1.00
Flag question

Se execută programul de mai jos, fiind inițializat cu

```
int n=6; int v[]={3, 9, 6, 1, 8, 6};

int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += v[i];
        if(v[i] % 4 == 1) {
            exit(0);
        }
    } else {
        sum += 12;
        if(v[i] % 4 == 2) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de 5 ori.
Valoarea minimă a sumei afișate este 7.
Valoarea maximă a sumei afișate este 36.

Your answer is incorrect.

Quiz navigation
1 2 3 4 5 6
Show one page at a time
Finish review

moodle-cs-utcn Home Dashboard My courses

x

Regulament de etică profes...

Cursuri - printer friendly

indrumator - temp

▼ Examen

Exam seria A, part 1; 15... 🔒

Exam seria A, part 2; 40... 

Exam seria B, part 1; 15 min...

Exam seria B, part 2; 40 min...

Exam, part 3; 40 min; 1... 

▼ 26 februarie - 3 martie

Curs 1: Intro

Curs 1: Quiz A

Curs 1: Quiz B

Lab 1: Instalarea și primii p...

Lab 1: Quiz

Question 6

Correct

Mark 1.00 out of 1.00

Flag question

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 2KB. Spațiul virtual de adrese al fiecărui proces este de 512KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 64KB.

Se dă tabela de paginare pentru un proces P , cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	6
3	5
5	1
6	8

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓

Numărul de cadre din memoria fizică a sistemului este: 32 ✓

Adresele virtuale în acest sistem de operare se pot reprezenta pe 19 biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
12200	4008 ✓
13159	17255 ✓
17000	-1 ✓
7524	11620 ✓